



객체지향 + PyQt5

김 승 환

swkim4610@inha.ac.kr

5남 107호

Algorithm Training

1. 최대, 최소값 구하기
2. 소트, 랭크 구하기(숫자, 문자)
3. 피보나치 수열 구하기
4. 피보나치 수열에 의해 사람수 별 치킨 수 추정하기
5. n 과 r 을 입력받아 nCr 계산하기
6. 수치적분/몬테칼로 적분하기

객체지향프로그래밍

- 프로그램 소스코드는 순차 실행의 종속성을 가지고 있어 프로그램 내용이 복잡 해지면 관리가 어려움
- 여러 사람이 공동으로 개발하기 어려움
- 윈도우 프로그램처럼 Top-Down 실행이 아닌 경우의 프로그램을 개발하고자 함
- 프로그램의 재활용 등 부품화, 모듈화가 필요해 짐
- 순차 실행 방식에서 프로그램의 재 사용을 위한 함수를 사용하였으나 객체 지향 프로그래밍에서는 재사용 및 여러 사람이 협업을 위하여 클래스를 사용함
- 클래스는 연관된 함수의 모음으로 예를 들어, mp3에 관련된 클래스에는 play, stop, next, before, volume up, volume down 등의 함수를 만들 수 있다. 즉, 연관 함수의 모음을 클래스라고 할 수 있음
- 파이썬은 이미 객체 지향 언어로 만들어져 있음

```
import math
print(math.sqrt(2))
Print(math.log(3))
```

객체지향프로그래밍

클래스는 객체의 일반화된 형태로 생성자, 필드, 메소드로 이루어져 있다.

객체는 클래스에서 생성자를 통해 초기 모양으로 생성하는 역할을 하고, 필드는 객체의 속성을 관리한다.

메소드는 객체가 수행하는 기능을 구현한 함수를 말한다.

아래의 프로그램에서 Describe는 클래스이고, stat는 Describe 클래스에서 생성된 객체다.

stat.mean(), stat.sd()는 각각 메소드다.

```
class Describe:
    def __init__(self, x):
        self.x = x
        self.n = len(x)
    def mean(self):
        summ = 0
        for i in range(self.n):
            summ += x[i]
        return summ / self.n
    def sd(self):
        summ = 0
        mean = self.mean()
        for i in range(self.n):
            summ += (x[i] - mean) ** 2
        return summ / (self.n - 1)
```

```
x = [1,2,3,4,5]
stat = Describe(x)
print(stat.mean(), stat.sd())
```

클래스변수와 객체변수

population은 클래스 변수로 Robot에 소속되어 있다. R2-D2, C-3PO는 droid1, droid2 이름의 객체로 각각 객체 변수 self.name을 가지고 있다. @classmethod 명령으로 how_many()가 클래스메소드임을 선언한다. 결과는 아래와 같다.

```
(Initializing R2-D2)
Greetings, my masters call me R2-D2.
We have 1 robots.
(Initializing C-3PO)
Greetings, my masters call me C-3PO.
We have 2 robots.
Robots can do some work here.
Robots have finished their work. So
let's destroy them.
R2-D2 is being destroyed!
There are still 1 robots working.
C-3PO is being destroyed!
C-3PO was the last one.
We have 0 robots.
```

```
class Robot:
    population = 0
    def __init__(self, name):
        self.name = name
        print ("({}) Initializing {}".format(self.name))
        Robot.population += 1
    def die(self):
        print ("{} is being destroyed!".format(self.name))
        Robot.population -= 1
        if Robot.population == 0:
            print ("{} was the last one.".format(self.name))
        else:
            print ("There are still {} robots working.".format(Robot.population))
    def say_hi(self):
        print ("Greetings, my masters call me {}".format(self.name))
    @classmethod
    def how_many(cls):
        print ("We have {} robots.".format(cls.population))
# class end
droid1 = Robot("R2-D2")
droid1.say_hi()
Robot.how_many() # Call class method
droid2 = Robot("C-3PO")
droid2.say_hi()
Robot.how_many()
print ("\nRobots can do some work here.\n")
print ("Robots have finished their work. So let's destroy them.")
droid1.die()
droid2.die()
Robot.how_many()
```

23_robotClass.py

상속

상속은 슈퍼클래스와 서브클래스의 개념으로 서브클래스는 슈퍼클래스로부터 상속받아 만든다. 이때, 서브클래스는 슈퍼클래스의 속성과 메소드를 그대로 사용할 수 있다.

SchoolMember는 슈퍼클래스이고, Teacher, Student는 서브클래스이다. SchoolMember에 정의된 이름과 나이 그리고 tell 메소드는 서브 클래스에서 상속되어 사용됨을 알 수 있다.

(Initialized Teacher: Mrs. Shrividya)

(Initialized Student: Swaroop)

Name:"Mrs. Shrividya" Age:"40" Salary: "30000"

Name:"Swaroop" Age:"25" Marks: "75"

24_subclassTest.py

```
class SchoolMember:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def tell(self):
        print ('Name:{} Age:{}".format(self.name, self.age))

class Teacher(SchoolMember):
    def __init__(self, name, age, salary):
        SchoolMember.__init__(self, name, age)
        self.salary = salary
        print ('(Initialized Teacher: {})'.format(self.name))
    def tell(self):
        SchoolMember.tell(self)
        print ('Salary: "{}:{}".format(self.salary)

class Student(SchoolMember):
    def __init__(self, name, age, marks):
        SchoolMember.__init__(self, name, age)
        self.marks = marks
        print ('(Initialized Student: {})'.format(self.name))
    def tell(self):
        SchoolMember.tell(self)
        print ('Marks: "{}:{}".format(self.marks))

t = Teacher('Mrs. Shrividya', 40, 30000)
s = Student('Swaroop', 25, 75)
members = [t, s]
for member in members:
    member.tell()
```

객체지향 연습-1

좌측 프로그램을 객체지향 방식으로 수정해보자. 클래스 이름은 fibonacci이고 생성인자는 n을 가진다.
수열을 만들기 위한 메소드로 getSeries() 를 만들었다.

```
# Fibonacci Series

numEnd=100
f=[]
f.append(0)
f.append(1)

i=2
fVal=0
while(fVal<=100):
    fVal=f[i-2]+f[i-1]
    f.append(fVal)
    i+=1

print f
```

```
class fibonacci:
    def __init__(self, n):
        self.n = n
    def getSeries(self):
        f=[]
        f.append(0)
        f.append(1)
        i=2
        fVal=0
        while(fVal<=self.n):
            fVal=f[i-2]+f[i-1]
            f.append(fVal)
            i+=1
        return f
# 100 이하의 피보나치수열을 만들어서 fSeries에 저장한다.
fSeries= fibonacci(100)
print (fSeries.getSeries())
```

객체지향 연습-2

1. 피보나치수열 클래스를 만들어 사람수에 따른 치킨 수를 구해보자.
2. 두 개의 정수 x, y 를 가진 Point클래스를 정의하고 두 개의 포인트 객체의 거리를 계산하는 정적 클래스를 만드시오.
3. 원 기둥은 반지름(r)과 높이 속성(h)을 가진다. 원기둥의 부피($\pi r^2 h$)와 겉넓이($2\pi r h + 2\pi r^2$)를 구하는 메소드를 작성하시오.
4. 리스트 자료에 대해 min, max, sort 결과를 제공하는 메소드를 가진 클래스를 작성하시오.
5. 년도, 월, 날을 주면 파이썬 날짜로 바꾸고, 그 날의 요일과 특정일에서 부터의 날짜수를 리턴하는 메소드를 만드시오.
6. 커피숍에서 들어오는 손님에 대해 입장시간, 전화번호, 거주지, 테이블 위치(1,2,3,4,T), 퇴장시간을 지정하는 클래스를 작성하시오. 그리고, 특정시간에 같이 있었던 손님의 리스트를 반환하는 프로그램을 작성하시오.

입출력(file I/O)

이 예제는 파일을 쓰기모드로 Open 하고, write 메소드로 텍스트를 저장한 다음 다시 읽기 모드로 읽어보는 예이다.

28_ioTest.py

```
poem = '''\
Programming is fun
When the work is done
if you wanna make your work also fun:
use Python!
'''

# Open for 'w'riting
f = open('d:/poem.txt', 'w')
# Write text to file
f.write(poem)
# Close the file
f.close()

# If no mode is specified,
# 'r'ead mode is assumed by default
f = open('d:/poem.txt')
while True:
    line = f.readline()
    # Zero length indicates EOF
    if len(line) == 0:
        break
    print (line, end="")
f.close()
```

입출력(pickle)

pickle 모듈은 파이썬 객체를 파일로 저장했다가 다시 불러 사용할 수 있도록 도와준다.

import 명령으로 pickle 모듈을 불러오고 리스트를 저장할 파일을 정한 다음 'wb' 모드로 엽니다.

여기서, wb는 write binary를 나타낸다. 이후, pickle.dump 메소드로 리스트를 파일에 write한다.

29_pickleTest.py

```
import pickle
# The name of the file where we will store the object
shoplistfile = 'shoplist.data'
# The list of things to buy
shoplist = ['apple', 'mango', 'carrot']
# Write to the file
f = open(shoplistfile, 'wb')
# Dump the object to a file
pickle.dump(shoplist, f)
f.close()
# Destroy the shoplist variable
del shoplist
# Read back from the storage
f = open(shoplistfile, 'rb')
# Load the object from the file
storedlist = pickle.load(f)
print(storedlist)
```

입출력(한글사용)

컴퓨터에서 영어 이외의 문자를 표현하기 위해서는 Encoding을 해야 한다.

영문자는 1바이트 2^8 개에 모든 문자의 표현이 가능하지만, 1 바이트로 표현이 불가능한 언어도 많이 있기 때문에 전 세계적으로 UniCode를 사용한다. UniCode의 종류로 utf-8은 ANSI 부분은 똑같이 1바이트, 유럽문자는 2바이트, 아시아문자는 3바이트 등으로 가변 표기하는 방법이다.

ANSI를 개조해서 영문은 1바이트, 한글은 2바이트로 변형한 것이 euc-kr이다. 문제는 윈도우에서는 euc-kr을 사용하고 리눅스, 유닉스, 오픈소스 환경에서는 utf-8을 사용해 윈도우에서 오픈소스 프로그램을 활용할 때, utf-8로 변환하는 것이 필요한 경우가 많다.

파이썬에서 한글로 파일을 쓰고, 읽기 위해서는 맨 앞줄에 `# encoding=utf-8` 으로 선언한 후, u” “ 형식으로 한글을 사용하면 된다.

30_unicodeTest.py

```
# encoding=utf-8
import io
f = io.open("abc.txt", "w", encoding="utf-8")
f.write(u"한글 테스트")
f.close()
text = io.open("abc.txt", encoding="utf-8").read()
print(text)
```

Exception

예외처리는 프로그램에서 있어야 할 파일이 없을 때, 데이터베이스가 꺼져 데이터를 불러올 수 없을 때 등 예기치 않은 오류가 발생할 때, 시스템 오류 대신 사용자 프로그램으로 오류를 처리하는 방식이다.

아래의 코드는 숫자를 입력 받는 경우에 숫자가 아닌 문자가 입력되었을 때 예외처리를 하는 방법이다.

```
a = input("숫자를 입력하세요")
try:
    a = int(a)
except:
    print("입력이 숫자가 아닙니다.")
else:
    print(a)
```

pyQT는 파이썬에서 GUI 프로그램을 도와주는 툴로 아나콘다에서는 이미 설치되어 있다.
아나콘다를 사용하지 않는 경우, 아래의 주소에서 설치할 수 있다.

<https://riverbankcomputing.com/software/pyqt/download>

파이썬 버전과 컴퓨터 비트 수에 따라 적절한 버전을 받아야 한다.

GUI 프로그램의 개발 단계는 UI Design → Event Listener 정의로 이루어진다.

GUI는 Sequential Program과 다르게 사용자의 마우스 클릭, 키보드 입력 등 사용자의 행동을 이벤트로 정의하고 이벤트가 발생할 경우, 어떤 Action을 수행해야 하는지를 프로그래밍한다.

HelloGUI

아래의 프로그램은 Hello World!를 출력하는 GUI 프로그램이다.

먼저, QtWidgets에 있는 QApplication, QWidget, QLabel을 import 하고, window()라는 함수를 정의한 다음 window()를 실행하는 방식이다. w는 윈도우를 나타내는 객체로 (100,100) 위치에 가로로 200, 세로로 50의 크기를 가진다.

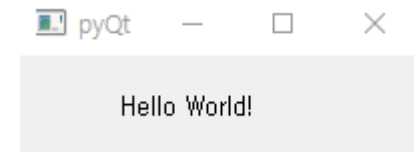
b는 QLabel이라는 위젯에 “Hello World”라고 쓰고 위치를 50,20 위치에 출력하는 예제이다.

/Anaconda2/Lib/site-packages/PyQt5/QtWidgets.pyd 파일은 C언어를 컴파일해서 만든 동적 라이브러리이다.

1_HelloGUI.py

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QLabel
def window():
    app = QApplication(sys.argv)
    w = QWidget()
    b = QLabel(w)
    b.setText("Hello World ~~~!")
    w.setGeometry(100, 100, 200, 50)
    b.move(50, 20)
    w.setWindowTitle("pyQt")
    w.show()
    sys.exit(app.exec_())

window()
```



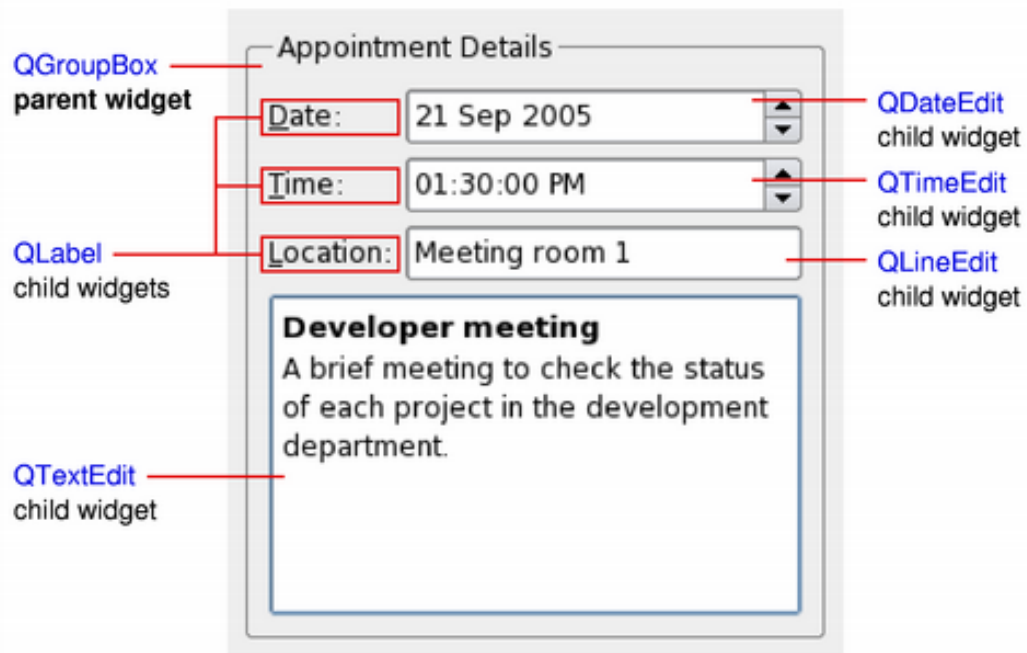
Widget

GUI 프로그램은 윈도우안에 아래와 같이 위젯을 배치하고 이들 간의 관계를 만들어 주는 방식이다.

위젯은 종류에 따라 QLabel, QDateTime, QTextEdit 등이 제공된다.

윈도우는 QMainWindow나 QDialog 클래스를 이용하여 객체로 만들 수 있다.

사실, 윈도우 역시 위젯이다. 위젯 중 최상위 위젯이다.



QMainWindow

class MyWindow는 QMainWindow를 상속받아 만든 객체이고 MyWindow를 생성할 때, 슈퍼클래스인 QMainWindow의 생성자를 실행한다.

2_QMainWindow.py

```
import sys
from PyQt5.QtWidgets import QMainWindow, QApplication

class MyWindow(QMainWindow):
    def __init__(self):
        super(MyWindow, self).__init__()
        self.setWindowTitle("QMainWindow")
        self.setGeometry(300, 300, 300, 400)

app = QApplication(sys.argv)
myWindow = MyWindow()
myWindow.show()
app.exec_()
```


QMainWindowEvent

아래는 이전 프로그램에 btn1이라는 QPushButton을 생성하고 QMessageBox를 이용해 “clicked”라는 메시지를 출력하는 예제이다.

3_QMainWindowEvent.py

```
import sys
from PyQt5.QtWidgets import QMainWindow, QApplication

class MyWindow(QMainWindow):
    def __init__(self):
        super(MyWindow, self).__init__()
        self.setWindowTitle(" QMainWindow Event")
        self.setGeometry(300, 300, 300, 400)
        btn1 = QPushButton(" Click me ", self)
        btn1.move(20, 20)
        btn1.clicked.connect(self.btn1_clicked)
    def btn1_clicked(self):
        QMessageBox.about(self, "message", "clicked")

app = QApplication(sys.argv)
myWindow = MyWindow()
myWindow.show()
app.exec_()
```

QDialogEvent

이 프로그램은 다이얼로그 윈도우(win)에 b1, b2의 QPushButton을 만들고 b1.clicked.connect(b1_clicked) 명령으로 b1이 클릭되었을 때, b1_clicked() 함수를 호출하도록 하는 프로그램이다.

4_buttonClick.py

```
import sys
from PyQt5.QtWidgets import QApplication, QDialog, QPushButton
def window():
    app = QApplication(sys.argv)
    win = QDialog()
    b1= QPushButton(win)
    b1.setText("Button1")
    b1.move(50,20)
    b1.clicked.connect(b1_clicked)
    b2=QPushButton(win)
    b2.setText("Button2")
    b2.move(50,50)
    win.setGeometry(100,100,200,100)
    win.setWindowTitle("PyQt")
    win.show()
    sys.exit(app.exec_())
```

```
def b1_clicked():
    print "Button 1 clicked"
def b2_clicked():
    print "Button 2 clicked"

window()
```

ui Designer

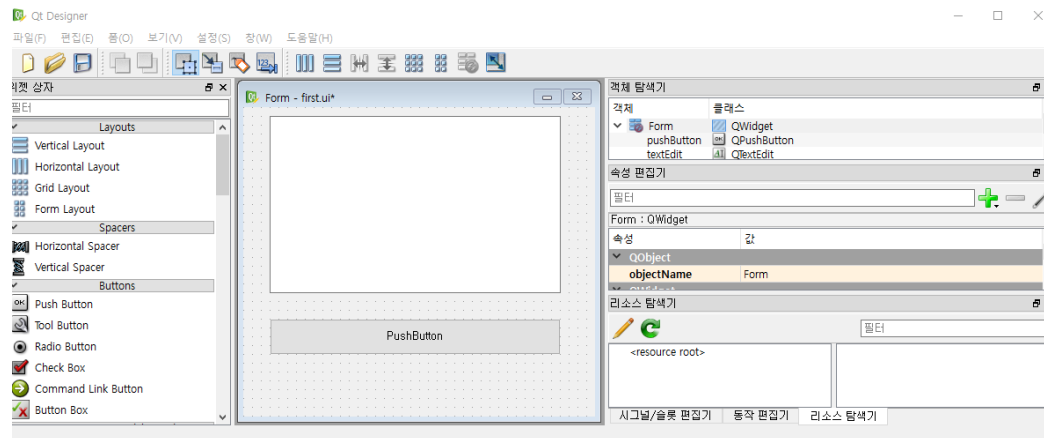
GUI 프로그래밍에서 화면을 디자인하는 것을 코드로 할 수도 있지만, 디자이너를 사용하면 좀 더 직관적으로 화면에서 파워포인트 작업 하듯이 수행할 수도 있다.

디자이너는 D:\ProgramData\Anaconda3\Library\bin와 같이 자신의 파이썬 설치 폴더에 designer.exe 파일이 있는데 이를 실행하면 QT 디자이너가 실행된다. QT 디자이너를 사용해 자신이 원하는 프레임을 디자인하고 저장을 하면 확장자가 ui인 파일이 생성된다. ui 파일은 xml 형식의 자료이다.

확장자가 ui 인 디자인 파일이 완성되면 *.ui를 파이썬 코드로 바꿔서 사용하는 방법과 *.ui를 파이썬 프로그램에서 읽어서 사용하는 방법이 있다.

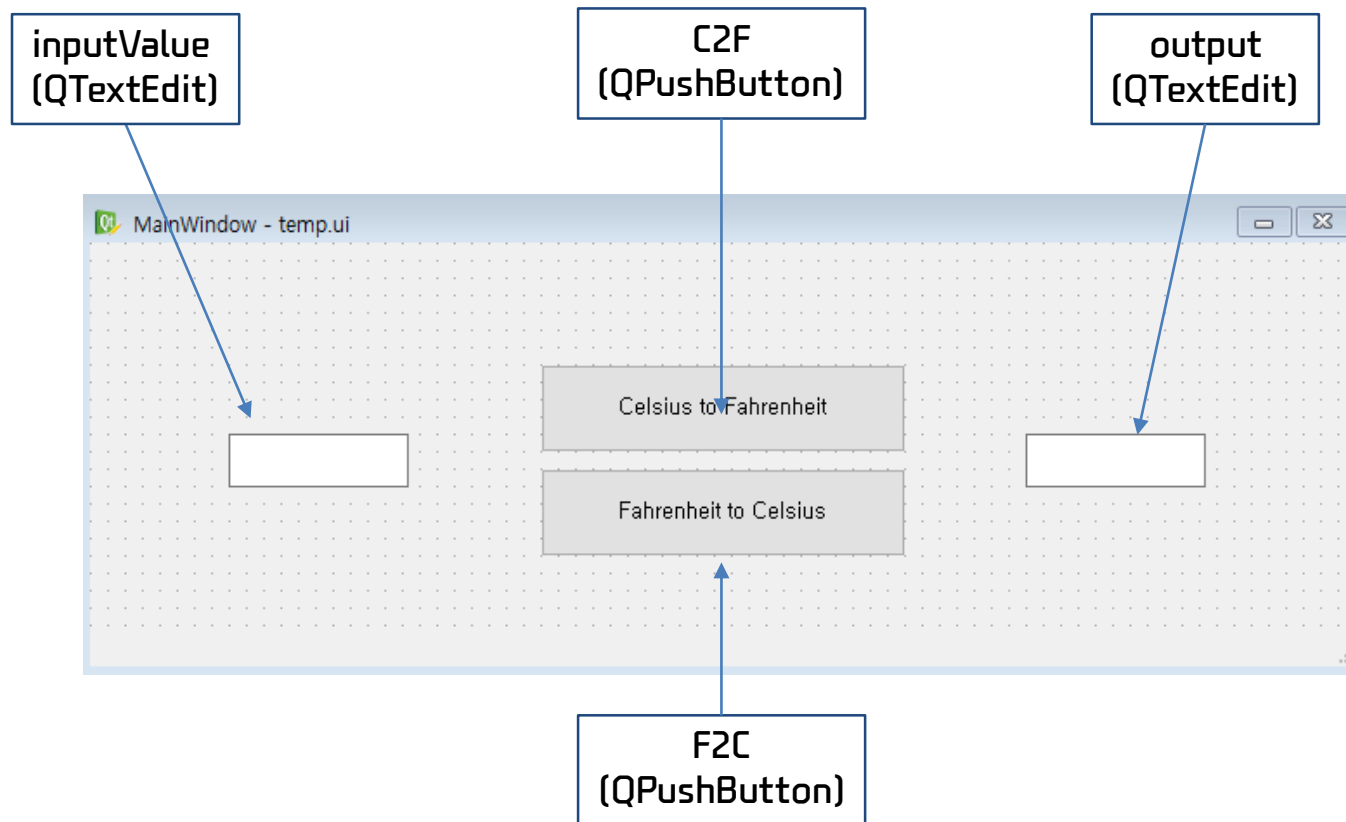
리눅스에서는 `sudo apt-get install qt4-designer` 명령으로 바이너리를 설치하면 된다.

바이너리는 `/usr/lib/x86_64-linux-gnu/qt4/bin/designer` 이다.



ui Designer

이제 ui 디자이너를 사용하여 아래와 같이 섭씨를 화씨로, 화씨를 섭씨로 변환하는 프로그램을 만들어 보자.



ui Designer

5_temp.py

```
import sys
from PyQt5.QtWidgets import QMainWindow, QApplication
from PyQt5.uic import loadUiType

form_class=loadUiType("temp.ui")[0]
class MyWindowClass(QMainWindow, form_class):
    def __init__(self, parent=None):
        QMainWindow.__init__(self, parent)
        self.setupUi(self)
        self.C2F.clicked.connect(self.C2F_clicked)
        self.F2C.clicked.connect(self.F2C_clicked)
    def C2F_clicked(self):
        cel=float(self.inputValue.toPlainText())
        fahr=cel*9/5.0+32
        self.output.setText(str(fahr))
    def F2C_clicked(self):
        fahr=float(self.inputValue.toPlainText())
        cel=(fahr-32)/9.0*5
        self.output.setText(str(cel))
app=QApplication(sys.argv)
myWindow=MyWindowClass(None)
myWindow.show()
app.exec_()
```

uic 모듈은 여러 기능을 하는데 대표적으로 Qt Designer를 통해서 저장했던 *.ui 파일들을 파이썬의 클래스에 불러들이는 기능이다. 리턴값은 두 개가 있는데 ui에 대한 레이아웃정보는 [0]번째에 있다. __init__ 메소드에서 C2F, F2C가 클릭되면 각각 C2F_clicked, F2C_clicked 를 실행하도록 정의하고, 각각의 함수에서 이벤트 발생 시, 해야 할 일을 프로그램 하였다.

SimpleCalc

qt Designer를 이용해서 우측과 같은 계산기 ui를 만들어 SimpleCalc.ui로 저장하자.

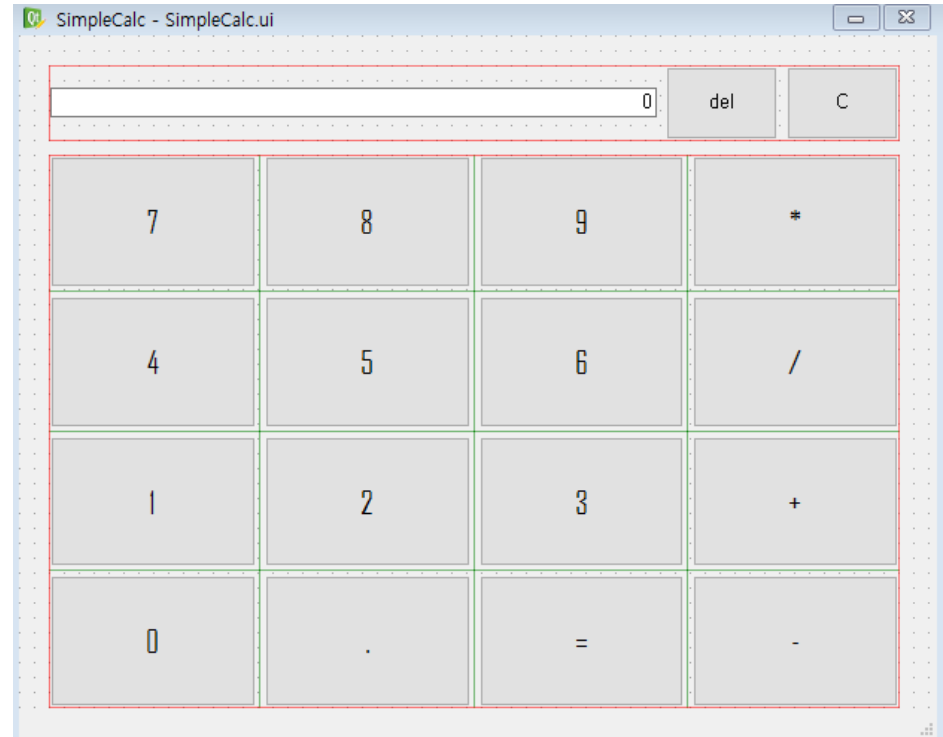
각 숫자는 b0, b1, ... , b9으로

del은 bDel, C는 bClear, 결과창은 QLineEdit 클래스의 result 객체이고 *,/,+,-는 각각 bMult, bDivide, bPlus, bMinus, .은 bDot의 이름을 부여하였다.

상단은 horizontal Layout으로 정렬하고,
하단은 grid Layout으로 정렬하였다.

이프로그램은 간단한 계산기로 12+3 과 같이 좌측, 우측에 숫자와 가운데 부분에 연산자가 있는 계산만 수행이 가능하다.

이 계산기에서 Simple을 제거하는 것은 여러분의 몫이다.



SimpleCalc

6_simpleCalc.py

```
import sys
from PyQt5.QtWidgets import QMainWindow, QApplication
from PyQt5.uic import loadUiType
form_class=loadUiType("SimpleCalc.ui")[0] # SimpleCalc.ui에서 객체 정보를 읽어온다.
# QMainWindow 클래스에서 상속받은 CalcClass 생성
class CalcClass(QMainWindow, form_class):
    def __init__(self, parent=None):
        QMainWindow.__init__(self,parent) # QMainWindow 클래스 객체 정보를 가져옴
        self.setupUi(self)
        nums = [self.b0, self.b1, self.b2, self.b3, self.b4, self.b5, self.b6, self.b7, self.b8, self.b9]
        # b0 ~ b9은 CalcClass global 객체이므로 self.b0로 표현
        for number in nums:
            number.clicked.connect(self.Nums) # 숫자가 입력되면 Nums 함수 실행
        self.bDel.clicked.connect(self.bDelClick)
        self.bClear.clicked.connect(self.bClearClick)
        self.bRun.clicked.connect(self.bRunClick)
        self.bPlus.clicked.connect(self.bPlusClick)
        self.bMinus.clicked.connect(self.bMinusClick)
        self.bMult.clicked.connect(self.bMultClick)
        self.bDivide.clicked.connect(self.bDivideClick)
        self.bDot.clicked.connect(self.bDotClick)
```

SimpleCalc

```
def Nums(self):
    global num
    sender = self.sender() # SimpleCalc 클래스의 sender 메소드를 통해 입력값을 전달받는다.
    newNum = int(sender.text())
    setNum = str(newNum)
    if self.result.text() == "0": # 입력값을 result 객체에 전달한다.(현재값이 "0"이면 0을 무시)
        self.result.setText(setNum)
    else:
        self.result.setText(self.result.text()+setNum)

def bDotClick(self):
    self.result.setText(self.result.text()+".")
def bPlusClick(self):
    self.result.setText(self.result.text()+"+")
def bMinusClick(self):
    self.result.setText(self.result.text()+"-")
def bDivideClick(self):
    self.result.setText(self.result.text()+"/")
def bMultClick(self):
    self.result.setText(self.result.text()+"*")
def bDelClick(self): # del이 클릭되면 result의 텍스트에서 마지막 부분을 잘라낸다.
    n=len(self.result.text())
    self.result.setText(self.result.text()[0:n-1])
def bClearClick(self):
    self.result.setText("0") # C가 클릭되면 result의 텍스트를 "0"으로 세팅
```


SimpleCalc

```
def bRunClick(self):
    A=self.result.text()
    for i in range(0,len(A)):
        # result를 한문자씩 읽으면서 숫자가 아니면 연산자라는 가정하에 연산자의 위치를 찾는다.
        if str(A[i]) == ".": continue
        if str(A[i]).isdigit() == False:
            # 연산자를 중심으로 왼쪽은 leftNum, 오른쪽은 rightNum에 숫자를 문자로 받아 실수로 변환
            leftNum=float(str(A[0:i]))
            rightNum=float(str(A[i+1:len(A)]))
            if str(A[i]) == "+": self.result.setText(str(leftNum+rightNum))
            if str(A[i]) == "-": self.result.setText(str(leftNum-rightNum))
            if str(A[i]) == "*": self.result.setText(str(leftNum*rightNum))
            if str(A[i]) == "/": self.result.setText(str(leftNum/rightNum))

app=QApplication(sys.argv)
myWindow=CalcClass(None)
myWindow.show()
app.exec_()
```

실행파일 만들기

파이썬은 Interpreter 방식의 언어이기 때문에 소스코드를 하나하나 실행한다.

이렇게 개발된 파이썬 프로그램을 배포하기 위해서는 배포판을 만들어야 하는데 방법은 여러 가지가 있다.

여기에서는 pyInstaller를 사용해보기로 한다.

pyInstaller를 설치하기 위해 시작화면에 Anaconda 메뉴에서 Anaconda Prompt를 실행하고 아래와 같이 명령하면 pyInstaller가 설치된다.

```
> pip install pyinstaller
```

이제, 여러분이 실행파일을 만들고자 하는 py 파일이 존재하는 폴더로 이동해서 아래와 같이 명령한다.

```
> pyinstaller --onefile --noconsole SimpleCalc.py
```

위의 명령이 실행되면 py 파일이 존재하는 폴더 밑에 dist라는 폴더가 생성되고 그 곳에 SimpleCalc.exe가 생성된다.

SimpleCalc.ui 등 파일이 실행되는데 필요한 파일을 dist 폴더로 복사해주면 실행될 것이다.

배포는 dist 폴더 밑에 있는 파일을 배포하면 된다.

도전과제

이제, 여러분이 스스로 작품을 만들어볼 차례이다. 주제는 수도쿠이다.

수도쿠는 1~9까지의 숫자를 9*9 행렬에 배열하는데 각 행과 열은 1~9까지의 숫자가 중복되어 나타날 수 없고 또 3*3 행렬 9개에서도 1~9까지의 숫자가 중복되어 나타나지 않도록 숫자를 배열하는 게임이다.

프로그램이 시작되면 각 셀이 $1 - 0.7 = 0.3$ 의 확률로 빈칸이 만들어진다. 사용자는 이 빈칸에 스도쿠 규칙에 적합한 1~9까지의 숫자를 채우고 Finish 버튼을 누르면 게임이 종료되는 것이다. Shuffle 버튼은 초기배열 값을 난수를 통해 재 생성하는 버튼이다. Shuffle 버튼은 가운데 확률값을 가지고 빈칸을 만드므로 0.7 보다 큰 값을 가지면 빈칸의 수가 작아져 난이도가 쉬워지고, 작은 값을 가지면 빈칸의 수가 많아져 난이도가 증가한다.

여러분은 먼저, QT Designer를 사용하여 우측 UI를 만들고 Sudoku.ui라는 이름으로 저장한다.



도전과제

아래는 81개의 버튼 Label 값이다. 버튼값을 이렇게 만드는 이유는 정답이 존재하는 수도쿠 배열을 만들기 위한 작전으로 먼저, 수도쿠 룰에 적합한 배열을 만들고 그 배열의 값을 랜덤하게 섞어 문제를 만들어 내기 위함이다.

123456789
456789123
789123456
231897564
564231897
897564231
312645978
645978312
978312645

즉, 1~9사이의 수에서 9개의 수를 비복원 추출한다. 예: [5, 2, 3, 9, 7, 4, 8, 1, 6]

그 다음으로 위의 배열에서 1→5, 2→2, 3→3, 4→9 와 같은 방식으로 숫자를 바꾼 다음 0.3의 확률로 각 셀 중에 빈칸을 만들어 문제를 출제하는 것이다. Shuffle 버튼을 누를 때 마다 새로운 문제가 출제된다.

도전과제

사용자는 원하는 버튼을 마우스로 클릭하여 어떤 셀에 값을 입력할 것인지 알려주고 키보드로 숫자를 입력한다.

그러면 그 숫자는 붉은 색으로 입력된다. 이렇게 모든 셀을 다 입력하고 Finish 버튼을 누르면 프로그램은

각 열에 숫자가 1~9까지 유일한가? 각 행의 숫자가 1~9까지 유일한가? 3*3 행렬의 값이 1~9까지 유일한가? 의 규칙을 검사하여 오류가 있는지 아니면 성공적으로 게임을 끝냈는지 결과를 출력하면 된다.



Sudoku

Shuffle

0.7

Finish

5	2	3	9	7	4	8	1	6
9	7	4	8	1	6	5	2	3
8	1	6	5	2	3	9	7	4
2	3	5	1	6	8	7	4	9
7	4	9	2	3	5	1	6	8
1	6	8	7	4	9	2	3	5
3	5	2	4	9	7	6	8	1
4	9	7	6	8	1	3	5	2
6	8	1	3	5	2	4	9	7

버튼을 클릭하고 1~9사이의 정수를 입력하세요. Finish를 누르면 채점 결과를 알려드립니다.

!!! ~~~Congratulation~~~ !!!

수고많았습니다.

이제 파이썬 프로그래머가

되어 보세요~~~